Detroit Transportation Survey

Multi-Stage Area Probability Sample Design

Effect Size

2026-04-11

1 Sample Design

1.1 Design Goals

The sample must satisfy four requirements simultaneously:

- Domain sample sizes: 1,000 respondents split equally (250 per group) across four age domains: 18–34, 35–49, 50–64, and 65+.
- Self-weighting within domains: every person in a given age group has the same overall selection probability, $\pi_k(d) = f_d$, so that no weighting adjustments are needed to compensate for unequal selection across PSUs within a domain.
- Equal workload per PSU: the total expected number of persons to be contacted is the same in every sample tract, simplifying field management and cost control.
- Two-stage selection: 40 tracts (PSUs) selected with probability proportional to a composite measure of size, and 1 BG (SSU) per tract, also selected PPS (MOS). Person-level sampling rates are specified for field implementation but not executed here.

1.1.1 Packages and setup

- Load the packages needed for data wrangling (`tidyverse`, `readxl`), sample selection (`sampling`), survey analysis (`survey`, `srvyr`, `PracTools`), spatial mapping (`sf`, `tigris`, `ggmap`, `tidycensus`, `viridis`), and report formatting (`patchwork`, `knitr`, `scales`). The `tidycensus` and `ggmap` packages each require an API key.

```
pacman::p_load(
  tidyverse, readxl,
  sampling,
  survey, srvyr,
  PracTools,
  sf, tigris, ggmap,
  tidycensus,
  viridis, patchwork,
  knitr, kableExtra, scales, here
)

options(tigris_class = "sf", tigris_use_cache = TRUE, scipen = 999)
set.seed(-77)
```

```r
# add api key, this will only work for me
# load private API keys
api_keys <- read_csv("D:/repos/api-keys.csv", show_col_types = FALSE)

# Census API key
census_key <- api_keys |>
  filter(key_id == "Census_key") |>
  pull(key)
census_api_key(key = census_key, install = FALSE)

# Stadia Maps API key
stadia_key <- api_keys |>
  filter(key_id == "stadiamaps_key") |>
  pull(key)
register_stadiamaps(key = stadia_key)
```

1.1.2 Data ingestion

- Quarto renders in a completely fresh R session and does not have access to objects loaded in the RStudio live environment. I therefore read the data files using explicit paths. The `semi_join()` on `tract_fips` restricts the Wayne County records to only those tracts that fall inside the City of Detroit boundary.

```r
# anchor to project root regardless of where the .qmd lives
here::i_am("inputs/Detroit_MI.xlsx")

input_path <- here("inputs")
census_tract <- "Census_Tract_FIPS_Code_Detroit_MI.csv"
detroit_mi <- "Detroit_MI.xlsx"

tract_fips <- read_csv(
  file.path(input_path, census_tract),
  col_names = "GEOID",
  col_types = cols(GEOID = col_character()),
  skip = 1,
  show_col_types = FALSE)

tract <- read_excel(
  file.path(input_path, detroit_mi),
  sheet = "Tract") |>
  mutate(Tract = as.character(Tract)) |>
  semi_join(tract_fips, by = c("Tract" = "GEOID")) |>
  mutate(TotPerson = rowSums(across(starts_with("Age"))))

bg <- read_excel(
  file.path(input_path, detroit_mi),
```

```r
  sheet = "BlockGroup") |>
  mutate(
    BlockGroup = as.character(BlockGroup),
    Tract = substr(BlockGroup, 1, 11)
  ) |>
  semi_join(tract_fips, by = c("Tract" = "GEOID"))
```

1.2 Target Population

The target population consists of all non-institutionalized persons aged 18 and older residing in the City of Detroit, Michigan. The 2020 Decennial Census DHC Summary File reports approximately 480,131 adults (18+) in approximately 309,916 households across 270 eligible census tracts. The average number of adults per household is approximately 1.55.

1.3 Area Sampling Frame

1.3.1 Data sources

The sampling frame is constructed from two files:

- **Census Tract FIPS Code Detroit MI.csv**: 280 FIPS codes identifying Detroit census tracts, from the City of Detroit Open Data Portal.
- **Detroit MI.xlsx**: tract- and block-group-level population data provided by the course instructors, originating from the 2020 Decennial Census DHC Summary File for Wayne County, MI. It includes housing unit counts, total person counts, and person counts by sex and age from the P12 table (Sex by Age).

1.3.2 Data cleaning

- Tract 26163985500 (tract 9855) appears identically 25 times in the data with zero population in every column, likely a water body or uninhabited industrial zone. Since it contributes no eligible population, I drop all instances.

```r
tract |>
  filter(Tract == "26163985500") |>
  select(1:10) |>
  head() |>
  kable(
    booktabs = TRUE,
    format = "latex"
  ) |>
  kable_styling(
    latex_options = c("striped", "scale_down", "hold_position")
  )
```

Tract	NAME	TotHH	TotPerson	TotMale	MaleLT5Yrs	Male5to9Yrs	Male10to14Yrs	Male15to17Yrs	Male18to19Yrs
26163985500	Census Tract 9855; Wayne County; Michigan	0	0	0	0	0	0	0	0
26163985500	Census Tract 9855; Wayne County; Michigan	0	0	0	0	0	0	0	0
26163985500	Census Tract 9855; Wayne County; Michigan	0	0	0	0	0	0	0	0
26163985500	Census Tract 9855; Wayne County; Michigan	0	0	0	0	0	0	0	0
26163985500	Census Tract 9855; Wayne County; Michigan	0	0	0	0	0	0	0	0
26163985500	Census Tract 9855; Wayne County; Michigan	0	0	0	0	0	0	0	0

```r
# drop this tract
tract <- tract |>
  filter(Tract != "26163985500")

cat("Tracts remaining:", tract |> nrow())
```

```
Tracts remaining: 275
```

2 Frame construction (Assignment 8)

- The DHC P12 table provides 49 Sex × Age columns per geography (e.g., `Male18to19Yrs`, `Female22to24Yrs`). I need to collapse these into the four survey age domains (18–34, 35–49, 50–64, 65+). The `age_patterns` named vector below is a single source of truth: it maps each domain label to a regex pattern that matches the corresponding P12 column names. If the domain boundaries ever change, only this vector needs updating.
- The pipeline works as follows: `pivot_longer()` reshapes all sex-by-age columns into rows, `map_chr()` classifies each row into a domain using the regex patterns, and `pivot_wider()` collapses back to one row per tract (or BG) with domain population counts as columns. Persons under 18 do not match any pattern and are removed by `filter(!is.na(AgeGroup))`.

```r
# single source of truth for age domain classification.
# each name is the output domain label; each value is a
# regex pattern matching the P12 column name fragments
# that belong to that domain.
age_patterns <- c(
  "Age18to34" = "18to19|20Yrs|21Yrs|22to24|25to29|30to34",
  "Age35to49" = "35to39|40to44|45to49",
  "Age50to64" = "50to54|55to59|60to61|62to64",
  "Age65plus" = "65to66|67to69|70to74|75to79|80to84|GE85")

# ssu frame (block-group-level): one row per BG with
# TotHH, TotAdult, and four age domain counts.
# this is the SINGLE SOURCE from which psu_frame is derived,
# guaranteeing S_i = sum_j(S_ij) at every subsequent step.
# (the Tract sheet is NOT used for population data because
# the 2020 DHC applies differential privacy noise independently
# at each geographic level, so tract totals != sum of BG totals.)
ssu_frame <- bg |>
  pivot_longer(
    cols = matches("^Male[^T]|^Female[^T]"),
    names_to = "SexAge", values_to = "elements") |>
  select(-TotPerson) |>
  mutate(
    AgeGroup = map_chr(SexAge, ~ {
      match <- names(age_patterns)[str_detect(.x, age_patterns)]
      if (length(match) == 0) NA_character_ else match})
  ) |>
  filter(!is.na(AgeGroup)) |>
  summarise(n = sum(elements), .by = c(BlockGroup, Tract, TotHH, AgeGroup)) |>
  pivot_wider(names_from = AgeGroup, values_from = n) |>
  mutate(TotAdult = rowSums(across(starts_with("Age")))) |>
  relocate(TotAdult, .after = "TotHH")

# psu frame (tract-level): aggregate UP from ssu_frame so that
# tract totals = sum of BG totals by construction
psu_frame <- ssu_frame |>
```

```r
  summarise(
    TotHH = sum(TotHH),
    TotAdult  = sum(TotAdult),
    Age18to34 = sum(Age18to34),
    Age35to49 = sum(Age35to49),
    Age50to64 = sum(Age50to64),
    Age65plus = sum(Age65plus),
    .by = Tract)

cat("psu frame:", nrow(psu_frame), "tracts\n")
```

```
psu frame: 276 tracts
```

```r
cat("ssu frame:", nrow(ssu_frame), "block groups\n")
```

```
ssu frame: 627 block groups
```

2.0.1 Quality control and linking (VDK Ch. 11, Sec. 11.1)

- Before sample selection, I need to verify that every unit in the frame can support the required workload. VDK Chapter 11 (Sec. 11.1) describes the quality control checks: (1) does each BG provide an adequate workload? (2) will any BG have a large selection probability? The key feasibility check from VDK p. 287 is that the within-BG sampling rate must not exceed 1:

$$\pi_{k|ij}(d) = \frac{\bar{q}\, f_d}{S_{ij}} \leq 1 \quad \text{for every BG } j \text{ and domain } d$$

  where $\bar{q}$ is the workload per BG, f_d is the domain sampling rate, and S_{ij} is the BG-level composite MOS. If a BG violates this, more persons would need to be selected from a domain than actually exist there.

- First, drop tracts and BGs with zero adult population (these have $S_i = 0$ and would cause division-by-zero in the selection probability formulas). Then, identify small tracts that have all their BGs violate the feasibility check, and combine them with geographic neighbors following a linking procedure introduced by Kish (1965) and recommended in VDK Ch. 11. A few small BGs within otherwise large tracts also violate feasibility, but since they have near-zero MOS they are extremely unlikely to be selected by the PPS procedure; note their existence and would apply Kish linking in the field if one were selected.

```r
# drop tracts with zero adult population, these are
# ineligible areas (water, industrial, institutional)
zero_tracts <- psu_frame |> filter(TotAdult == 0)
cat("Zero-adult tracts to drop:\n")
```

```
Zero-adult tracts to drop:
```

```r
zero_tracts |> select(Tract, TotHH, TotAdult) |> kable()
```

Tract	TotHH	TotAdult
26163981700	0	0
26163981800	1	0

Tract	TotHH	TotAdult
26163984200	0	0
26163985500	0	0
26163985800	0	0
26163985900	0	0

```
psu_frame <- psu_frame |> filter(TotAdult > 0)
ssu_frame <- ssu_frame |> filter(Tract %in% psu_frame$Tract)

# also drop zero-adult BGs within remaining tracts
cat("Zero-adult BGs dropped:", sum(ssu_frame$TotAdult == 0), "\n")
```

```
Zero-adult BGs dropped: 9
```

```
ssu_frame <- ssu_frame |> filter(TotAdult > 0)

# rebuild psu_frame from surviving BGs so totals stay consistent
psu_frame <- ssu_frame |>
  summarise(
    TotHH = sum(TotHH),
    TotAdult  = sum(TotAdult),
    Age18to34 = sum(Age18to34),
    Age35to49 = sum(Age35to49),
    Age50to64 = sum(Age50to64),
    Age65plus = sum(Age65plus),
    .by = Tract)

cat("After dropping zeros:", nrow(psu_frame), "tracts,",
    nrow(ssu_frame), "BGs\n")
```

```
After dropping zeros: 270 tracts, 612 BGs
```

- Next, compute preliminary design parameters to identify undersized units. The domain sampling rates under Approach A (selection-based, see Design Parameter section below) are $f_d = n_d^{\text{select}}/N_d$ where $n_d^{\text{select}} = 250/r_d$ and r_d is the expected response rate from the project assignment. The workload per BG is $\bar{q} = \sum_d n_d^{\text{select}}/(m \cdot \bar{n})$. The minimum BG MOS for feasibility is $\bar{q} \cdot \max(f_d)$, corresponding to roughly 62 adults depending on age mix.

```
# design constants from the assignment
m <- 40       # sample PSUs
n_bar <- 1    # BGs per sample PSU

# preliminary f_d from the current (pre-linking) frame
prelim_params <- tibble(
  domain = c("18-34", "35-49", "50-64", "65+"),
  n_respond = rep(250, 4),
```

```r
  # given to us
  resp_rate = c(0.35, 0.45, 0.50, 0.65),
  N_d = c(sum(psu_frame$Age18to34), sum(psu_frame$Age35to49),
          sum(psu_frame$Age50to64), sum(psu_frame$Age65plus))) |>
  mutate(n_select = n_respond / resp_rate, f_d = n_select / N_d)

f_d_prelim <- prelim_params$f_d
names(f_d_prelim) <- prelim_params$domain
q_bar_prelim <- sum(prelim_params$n_select) / (m * n_bar)

# minimum mos needed for feasibility: q_bar * max(f_d)
min_mos <- q_bar_prelim * max(f_d_prelim)
cat("Min BG MOS for feasibility:", round(min_mos, 4), "\n")
```

```
Min BG MOS for feasibility: 0.2642
```

```r
# compute preliminary MOS inline for feasibility checks only.
# do NOT write MOS_bg onto ssu_frame, the final MOS_bg
# must be computed later with the post-linking f_d.
prelim_mos <- ssu_frame |>
  mutate(MOS_prelim = f_d_prelim["18-34"] * Age18to34 +
           f_d_prelim["35-49"] * Age35to49 +
           f_d_prelim["50-64"] * Age50to64 +
           f_d_prelim["65+"] * Age65plus)

# identify violating BGs
ssu_violations <- prelim_mos |> filter(MOS_prelim < min_mos)
cat("How many BGs are failing feasibility?", nrow(ssu_violations), "\n\n")
```

```
How many BGs are failing feasibility? 14
```

```r
# which tracts have ALL their BGs failing? these are the
# undersized tracts that need PSU-level linking
tiny_tracts <- prelim_mos |>
  filter(Tract %in% ssu_violations$Tract) |>
  group_by(Tract) |>
  summarise(n_bgs = n(),
            n_fail = sum(MOS_prelim < min_mos), .groups = "drop") |>
  filter(n_fail == n_bgs) |>
  pull(Tract)

cat("How many undersized tracts (all BGs fail):", length(tiny_tracts), "\n")
```

```
How many undersized tracts (all BGs fail): 5
```

```r
psu_frame |> filter(Tract %in% tiny_tracts) |>
  select(Tract, TotHH, TotAdult) |>
  kable()
```

Tract	TotHH	TotAdult
26163524500	39	71
26163983600	3	9
26163984100	2	3
26163985000	8	16
26163985200	1	1

- Combine the undersized tracts with their nearest geographic neighbor on the FIPS-sorted frame. The combined unit's populations and MOS are summed. This is a simplified version of the Kish (1965) linking procedure: sort the frame, and for each undersized unit, merge it forward into the next adequately-sized unit.

```
# find the nearest non-tiny neighbor for each undersized tract
all_sorted <- psu_frame |> arrange(Tract) |> pull(Tract)
non_tiny <- setdiff(all_sorted, tiny_tracts)

psu_link_map <- tibble(original = tiny_tracts) |>
  mutate(linked_to = map_chr(original, function(tr) {
    fwd <- non_tiny[non_tiny > tr]
    bwd <- non_tiny[non_tiny < tr]
    if (length(fwd) > 0) fwd[1] else tail(bwd, 1)})
    )

cat("Linked PSUs:\n")
```

```
Linked PSUs:
```

```
psu_link_map |> kable()
```

original	linked_to
26163524500	26163524600
26163983600	26163985100
26163984100	26163985100
26163985000	26163985100
26163985200	26163985300

```
# update Tract in ssu_frame: linked tracts get their neighbor's id
ssu_frame <- ssu_frame |>
  mutate(Tract = if_else(
    Tract %in% psu_link_map$original,
    psu_link_map$linked_to[match(Tract, psu_link_map$original)],
    Tract))

# rebuild psu_frame from ssu_frame (the single source of truth)
psu_frame <- ssu_frame |>
  summarise(
    TotHH = sum(TotHH),
    TotAdult  = sum(TotAdult),
    Age18to34 = sum(Age18to34),
    Age35to49 = sum(Age35to49),
    Age50to64 = sum(Age50to64),
    Age65plus = sum(Age65plus),
    .by = Tract)

cat("How many PSUs after linking?", nrow(psu_frame), "PSUs\n")
```

```
How many PSUs after linking? 265 PSUs
```

```r
cat("How many combined PSUs?", n_distinct(psu_link_map$linked_to), "\n")
```

```
How many combined PSUs? 3
```

3 Design parameters (Assignment 8)

- The composite MOS method (VDK Sec. 10.5) requires domain sampling rates $f_d = n_d/N_d$, where n_d is the desired domain sample size and N_d is the domain population. VDK p. 282 defines this notation. There are two conventions for what n_d means:
  - **Approach A (selection-based)**: $n_d = n_d^{\text{select}} = 250/r_d$, the number of persons to *contact* in each domain, inflated for the expected non-response rate r_d. This builds NR into the MOS so the design directly yields enough selections to produce 250 respondents per domain after expected losses.
  - **Approach B (respondent-based, VDK Ch. 11 pattern)**: $n_d = 250$, the respondent target. This is how the Anne Arundel County example in VDK Table 11.1 is set up. NR inflation is then applied at the person-selection stage in the field rather than in the MOS.
- Under Approach A, the workload per BG is $\bar{q} = \sum_d n_d^{\text{select}}/(m \cdot \bar{n}) \approx 53.9$ persons to contact. Under Approach B, $\bar{q} = 1000/40 = 25$ respondents. I proceed with **Approach A** because it keeps all selection probabilities self-contained at the design stage; field teams simply apply the rates with no additional NR inflation needed.

```r
# recompute N_d from the cleaned, linked frame
N_d <- c(sum(psu_frame$Age18to34), sum(psu_frame$Age35to49),
         sum(psu_frame$Age50to64), sum(psu_frame$Age65plus))

# approach A: selection-based f_d
design_params <- tibble(
  domain = c("18-34", "35-49", "50-64", "65+"),
  n_respond = rep(250, 4),
  resp_rate = c(0.35, 0.45, 0.50, 0.65),
  N_d = N_d) |>
  mutate(
  n_select = n_respond / resp_rate, # inflate for NR
  f_d = n_select / N_d # domain sampling rate
)

cat("Design parameters (Approach A):\n")
```

```
Design parameters (Approach A):
```

```r
design_params |>
  mutate(across(where(is.numeric), ~ round(.x, 6))) |>
  kable()
```

domain	n_respond	resp_rate	N_d	n_select	f_d
18-34	250	0.35	161357	714.2857	0.004427
35-49	250	0.45	113272	555.5556	0.004905
50-64	250	0.50	117763	500.0000	0.004246
65+	250	0.65	87739	384.6154	0.004384

```r
# extract as named vector
f_d <- design_params$f_d
names(f_d) <- design_params$domain
total_select <- sum(design_params$n_select)
q_bar <- total_select / (m * n_bar)

cat("Total selections (S):", round(total_select, 1), "\n")
```

```
Total selections (S): 2154.5
```

```r
cat("Workload per BG (q_bar):", round(q_bar, 1), "persons\n")
```

```
Workload per BG (q_bar): 53.9 persons
```

4 Composite MOS (VDK Eq. 10.1, Assignment 8)

- The composite measure of size for PSU i is defined as (VDK Eq. 10.1, p. 282):

$$S_i = \sum_{d=1}^{D} f_d \cdot N_i(d)$$

  This quantity represents the expected sample size from PSU i if the desired domain sampling rates were applied to its domain populations. The total MOS across all PSUs equals the total desired sample size: $S = \sum_{i \in U} S_i = \sum_d f_d N_d = n^{\text{select}}$ (VDK p. 282).

- PSU selection probabilities are proportional to S_i (VDK p. 282):

$$\pi_i = \frac{m \cdot S_i}{S}$$

- For the three-stage extension (VDK p. 286), the BG-level composite MOS is $S_{ij} = \sum_d f_d \cdot Q_{ij}(d)$ where $Q_{ij}(d)$ is the population of domain d in BG j of tract i. The BG conditional selection probability is $\pi_{j|i} = \bar{n} \cdot S_{ij}/S_i$ (VDK p. 286). Note that since $\bar{n} = 1$ in this design, $\pi_{j|i} = S_{ij}/S_i$.

- The within-BG person sampling rate for domain d is (VDK p. 288):

$$\pi_{k|ij}(d) = \frac{\bar{q}\, f_d}{S_{ij}}$$

  The self-weighting property then follows algebraically (VDK p. 286):

$$\pi_i \cdot \pi_{j|i} \cdot \pi_{k|ij}(d) = \frac{m\, S_i}{S} \cdot \frac{\bar{n}\, S_{ij}}{S_i} \cdot \frac{\bar{q}\, f_d}{S_{ij}} = \frac{m\, \bar{n}\, \bar{q}}{S} f_d = f_d$$

  since $S = m \cdot \bar{n} \cdot \bar{q}$ (VDK p. 286).

- The workload is constant in every BG (VDK p. 287): $\sum_d q^*_{ij}(d) = \bar{q}$, where $q^*_{ij}(d) = \bar{q}\, f_d\, Q_{ij}(d)/S_{ij}$ is the expected number of domain-d selections in BG ij (analogous to VDK Eq. 10.2 at the SSU level).

```r
# psu-level composite mos and selection probabilities
domain_cols <- c("Age18to34", "Age35to49", "Age50to64", "Age65plus")

# compute MOS using explicit named multiplication to avoid
# any ambiguity in column-to-rate alignment
psu_frame <- psu_frame |>
  mutate(
    MOS = f_d["18-34"] * Age18to34 + f_d["35-49"] * Age35to49 +
          f_d["50-64"] * Age50to64 + f_d["65+"]   * Age65plus,
    pi_i = m * MOS / sum(MOS))

S_total <- sum(psu_frame$MOS)

cat("S =", round(S_total, 2),
```

```
    "| should equal total_select =", round(total_select, 2), "\n")
```

```
S = 2154.46 | should equal total_select = 2154.46
```

```
stopifnot(abs(S_total - total_select) < 1e-6)

cat("pi_i range:", round(range(psu_frame$pi_i), 4), "\n")
```

```
pi_i range: 0.0137 0.3629
```

```
cat("Any pi_i > 1?", any(psu_frame$pi_i > 1), "\n")
```

```
Any pi_i > 1? FALSE
```

```
# bg-level composite mos using the SAME f_d
ssu_frame <- ssu_frame |>
  mutate(
    MOS_bg = f_d["18-34"] * Age18to34 + f_d["35-49"] * Age35to49 +
             f_d["50-64"] * Age50to64 + f_d["65+"]   * Age65plus)

# consistency check: sum of BG MOS must equal PSU MOS
# (guaranteed because psu_frame is aggregated from ssu_frame)
ssu_frame |>
  summarise(sum_bg_mos = sum(MOS_bg), .by = Tract) |>
  left_join(psu_frame |> select(Tract, MOS), by = "Tract") |>
  mutate(diff = abs(sum_bg_mos - MOS)) |>
  pull(diff) |>
  {\(x) stopifnot(all(x < 1e-6))}()
```

4.1 VDK 10.1 verification table

- Following the spreadsheet layout in VDK Example 10.1 (Table 10.1, p. 284), verify four properties at the PSU level. These are the same checks from Assignment 8 (VDK Exercise 10.2) using the two-domain example, extended here to four domains and applied to the Detroit frame.

  1. **Feasibility**: $f_d \leq S_i/\bar{q}$ for every domain, equivalently $\bar{q} f_d / S_i \leq 1$.
  2. **Expected domain sample sizes**: $n_i^*(d) = \bar{q}\, f_d\, N_i(d)/S_i$ (VDK Eq. 10.2 adapted to this notation).
  3. **Constant workload**: $\sum_d n_i^*(d) = \bar{q}$ for every PSU.
  4. **Self-weighting**: $\pi_i \cdot \pi_{k|i}(d) = f_d$, where $\pi_{k|i}(d) = \bar{q}\, f_d / S_i$ is the within-PSU person rate.

```
vdk_table <- psu_frame |>
  # step 1: within-psu rates
  mutate(
    across(all_of(domain_cols),
           ~ q_bar * f_d[match(cur_column(), domain_cols)] / MOS,
           .names = "rate_{.col}")) |>
  # step 2: checks
  mutate(
```

```r
    across(starts_with("rate_"), ~ .x <= 1, .names = "feas_{.col}"),
    exp_workload = across(all_of(domain_cols),
                          ~ .x * q_bar *
                            f_d[match(cur_column(), domain_cols)] / MOS) |>
      rowSums(),
    across(starts_with("rate_"), ~ pi_i * .x, .names = "sw_{.col}")
    )

cat("(1) Are all within-psu rates <= 1?\n")
```

```
(1) Are all within-psu rates <= 1?
```

```r
vdk_table |> summarise(across(starts_with("feas"), ~ all(.x))) |>
  kable()
```

feas_rate_Age18to34	feas_rate_Age35to49	feas_rate_Age50to64	feas_rate_Age65plus
TRUE	TRUE	TRUE	TRUE

```r
cat("\n(3) workload per psu (should all =", round(q_bar, 2), "):\n")
```

```
(3) workload per psu (should all = 53.86 ):
```

```r
summary(vdk_table$exp_workload)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  53.86   53.86   53.86   53.86   53.86   53.86 
```

```r
cat("\n(4) self-weighting check (first 5 PSUs):\n")
```

```
(4) self-weighting check (first 5 PSUs):
```

```r
cat("target f_d:", round(f_d, 6), "\n")
```

```
target f_d: 0.004427 0.004905 0.004246 0.004384
```

```r
vdk_table |> select(Tract, starts_with("sw_")) |>
  slice_head(n = 5) |>
  mutate(across(where(is.numeric), ~ round(.x, 6))) |>
  kable()
```

Tract	sw_rate_Age18to34	sw_rate_Age35to49	sw_rate_Age50to64	sw_rate_Age65plus
26163533800	0.004427	0.004905	0.004246	0.004384
26163533900	0.004427	0.004905	0.004246	0.004384
26163534100	0.004427	0.004905	0.004246	0.004384
26163534200	0.004427	0.004905	0.004246	0.004384
26163534300	0.004427	0.004905	0.004246	0.004384

4.2 Uniform rate discussion

- Could a single sampling rate $f = n^{\text{select}}/N_{\text{adult}}$ be used for all persons regardless of age domain?
  - The table below shows the expected number of respondents per domain under such a uniform rate. Because response rates vary substantially (0.35 for ages 18–34 vs. 0.65 for 65+), a uniform rate over-samples domains with higher response rates and under-samples those with lower rates. The 35–49 group falls approximately 8.5% short of the 250 target. The composite MOS avoids this problem by building domain-specific rates into the design from the start.

```
f_uniform <- total_select / sum(psu_frame$TotAdult)
design_params |>
  mutate(exp_respond = f_uniform * N_d * resp_rate,
         shortfall_pct =
           round((n_respond - exp_respond) / n_respond * 100, 1)) |>
  select(domain, N_d, exp_respond, shortfall_pct) |>
  kable()
```

domain	N_d	exp_respond	shortfall_pct
18-34	161357	253.4159	-1.4
35-49	113272	228.7247	8.5
50-64	117763	264.2146	-5.7
65+	87739	255.9081	-2.4

5 Sample Selection (Assignment 5)

- Select the sample in two stages using `UPsystematic()` from the `sampling` package (Tille and Matei, 2016), which implements Madow's systematic PPS method. The PSU frame is sorted by FIPS code before selection, which provides implicit geographic stratification since adjacent tracts on the sorted list tend to be geographic neighbors, so the systematic selection spreads the sample across the city (VDK Sec. 3.3).
- The assignment specifies `set.seed(-77)` for reproducibility. Systematic PPS does not permit computation of exact joint selection probabilities ($\pi_{ii'} = 0$ for some pairs), which precludes the Horvitz-Thompson variance estimator. In practice, I use the with-replacement variance approximation via paired analytic strata.

5.1 Stage 1: PSU selection

- I select $m = 40$ tracts with inclusion probabilities $\pi_i = m \cdot S_i/S$. The `UPsystematic()` function takes these as input and returns a 0/1 indicator for each PSU.

```
# sort by fips for implicit geographic stratification
psu_frame <- psu_frame |>
  arrange(Tract)

# set.seed(-77) is called once in the setup chunk
# to govern the entire RNG stream for both stages
psu_sample <- psu_frame |>
  mutate(selected = UPsystematic(pik = pi_i)) |>
  filter(selected == 1)

cat("PSUs selected:", nrow(psu_sample), "\n")
```

```
PSUs selected: 40
```

5.2 Stage 2: SSU (block group) selection

- Within each of the 40 sample tracts, I select $\bar{n} = 1$ BG with probability proportional to the BG-level composite MOS: $\pi_{j|i} = \bar{n} \cdot S_{ij}/S_i = S_{ij}/S_i$ (VDK p. 286). For tracts with only a single BG, the selection is deterministic ($\pi_{j|i} = 1$). The `select_one_bg()` helper handles both cases.

```
# get BGs in sample tracts and compute conditional probabilities
bg_in_sample <- ssu_frame |>
  filter(Tract %in% psu_sample$Tract) |>
  left_join(psu_sample |>
              select(Tract, MOS_psu = MOS), by = "Tract") |>
  mutate(pij_given_i = n_bar * MOS_bg / MOS_psu)

# QC: conditional probs must sum to n_bar within each tract
bg_in_sample |>
  summarise(pij_sum = sum(pij_given_i), .by = Tract) |>
  pull(pij_sum) |>
```

```r
  {\(x) stopifnot(all(abs(x - n_bar) < 1e-6))}()

# helper: select 1 bg from a tract via systematic pps
select_one_bg <- function(bg_sub) {
  if (nrow(bg_sub) == 1) return(bg_sub |>
                                  mutate(ssu_selected = 1L))
  # pij_given_i sums to n_bar = 1 within each tract,
  # so it is directly the inclusion probability vector
  bg_sub |>
    mutate(ssu_selected = as.integer(
      UPsystematic(pik = pij_given_i)))
}

bg_selected <- bg_in_sample |>
  group_by(Tract) |>
  group_split() |>
  map_dfr(select_one_bg) |>
  filter(ssu_selected == 1)

cat("How many BGs selected?", nrow(bg_selected), "\n")
```

```
How many BGs selected? 40
```

5.3 Selection probabilities

- The joint selection probability of BG j in tract i is $\pi_{ij} = \pi_i \cdot \pi_{j|i}$ (VDK p. 286, Ch. 10). This is what the two-stage selection actually produces. I also compute the person-level domain rates $\pi_{k|ij}(d) = \bar{q} f_d / S_{ij}$ that field teams would implement, and verify that the expected workload per BG equals $\bar{q}$.

```r
sample_bgs <- bg_selected |>
  left_join(psu_sample |> select(Tract, pi_i), by = "Tract") |>
  mutate(
    # joint bg selection probability (the two-stage output)
    pi_ij = pi_i * pij_given_i,
    # person-level rates for field use (VDK p. 287)
    across(all_of(domain_cols),
           ~ q_bar * f_d[match(cur_column(), domain_cols)] / MOS_bg,
           .names = "rate_{.col}"),
    # expected workload
    exp_workload = across(all_of(domain_cols),
                          ~ .x * q_bar * f_d[match(cur_column(),
                                                   domain_cols)] / MOS_bg) |>
      rowSums()
  )

cat("What is the pi_ij range?", round(range(sample_bgs$pi_ij), 4), "\n")
```

```
What is the pi_ij range? 0.0365 0.1652
```

```r
cat("Are there any pi_ij > 1?", any(sample_bgs$pi_ij > 1), "\n")
```

```
Are there any pi_ij > 1? FALSE
```

```r
cat("Workload per BG (should =", round(q_bar, 1), "):\n")
```

```
Workload per BG (should = 53.9 ):
```

```r
summary(sample_bgs$exp_workload)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  53.86   53.86   53.86   53.86   53.86   53.86 
```

5.4 Description of household and person selection (third stage)

- In area probability sampling, persons are reached through households (VDK Ch. 10). I do not execute this stage because no address listing exists, but I describe the procedure that field teams would follow in each selected BG.
- The key outcome variables (vehicle ownership, transit use, skipping trips, asking for rides) are household-level or household-correlated characteristics. If multiple persons from the same household were selected, their responses would be correlated, inflating the design effect and reducing precision. For this reason I recommend selecting one eligible adult per sampled household using a random selection method such as the Kish grid or next-birthday rule.
- The field procedure within each selected BG:
  1. Household listing: interviewers compile a complete listing of all occupied housing units using maps and Census address data.
  2. Household sampling: households are sampled systematically at a rate that, combined with π_{ij}, yields the desired person-level rates. With approximately 1.55 eligible adults per household in Detroit, the HH rate is adjusted accordingly.
  3. Screening: at each sampled HH, interviewers administer a brief screener to enumerate all persons 18+ and classify each into an age domain.
  4. Within-HH selection: one eligible adult is selected at random. The selection probability is set to maintain $\pi_{k|ij}(d) = \bar{q} f_d / S_{ij}$ for each domain d.
  5. Verification: the overall probability is $\pi_i \cdot \pi_{j|i} \cdot \pi_{k|ij}(d) = f_d$, confirming self-weighting. The expected sample sizes are not integers, as VDK notes after Example 10.1, it is important to sample at the specified rates rather than rounding to fixed counts.

6 Anticipated Precision (Assignment 6)

- For the key survey variable (the proportion of Detroit residents whose main means of transportation is public transit), precision under the cluster design depends on the design effect. For a two-stage cluster sample, the approximate design effect is (VDK Ch. 9, Eq. 9.5):

$$\text{deff} \approx 1 + (\bar{b} - 1)\,\delta$$

  where $\bar{b}$ is the average number of respondents per PSU (≈ 25 in this design) and δ is the intraclass correlation coefficient measuring homogeneity within tracts. Values of δ for demographic/behavioral variables in U.S. urban populations typically range from 0.01 to 0.05, yielding deffs between 1.2 and 2.2.

- The variance of an estimated proportion $\hat{p}$ under the cluster design is approximately:

$$\text{Var}(\hat{p}) \approx \text{deff} \cdot \frac{p(1-p)}{n}$$

  An analyst would estimate this using the ultimate cluster variance estimator, which in the with-replacement form is:

$$v_{PSU} = \frac{m}{m-1} \sum_{i \in s} \left(\sum_{k \in s_i} w_k y_k - \frac{\sum_{i \in s} \sum_{k \in s_i} w_k y_k}{m} \right)^2$$

  This is implemented automatically by `survey::svymean()` when the design object specifies PSUs and strata (see Sec. 6).

- The table below shows the standard error, coefficient of variation, and 95% margin of error for a range of proportions and design effects at $n = 1000$ total respondents.

```r
expand_grid(p = c(.10, .15, .20, .25, .30),
            deff = c(1.5, 2.0, 2.5)) |>
  mutate(n = 1000,
         se = sqrt(deff * p * (1 - p) / n),
         cv = se / p,
         moe = 1.96 * se) |>
  mutate(across(where(is.numeric), ~ round(.x, 4))) |>
  kable(n = Inf)
```

p	deff	n	se	cv	moe
0.10	1.5	1000	0.0116	0.1162	0.0228
0.10	2.0	1000	0.0134	0.1342	0.0263
0.10	2.5	1000	0.0150	0.1500	0.0294
0.15	1.5	1000	0.0138	0.0922	0.0271
0.15	2.0	1000	0.0160	0.1065	0.0313
0.15	2.5	1000	0.0179	0.1190	0.0350
0.20	1.5	1000	0.0155	0.0775	0.0304
0.20	2.0	1000	0.0179	0.0894	0.0351
0.20	2.5	1000	0.0200	0.1000	0.0392
0.25	1.5	1000	0.0168	0.0671	0.0329

p	deff	n	se	cv	moe
0.25	2.0	1000	0.0194	0.0775	0.0380
0.25	2.5	1000	0.0217	0.0866	0.0424
0.30	1.5	1000	0.0177	0.0592	0.0348
0.30	2.0	1000	0.0205	0.0683	0.0402
0.30	2.5	1000	0.0229	0.0764	0.0449

- For a transit use proportion of approximately $p = 0.20$ with deff = 2.0, the 95% margin of error is $\pm$ 0.035, which is adequate for most policy analyses. Domain-level estimates (250 respondents each) would have approximately double the margin of error, around $\pm$ 0.07.

7 Weighting Plan (Assignment 9)

7.1 Overview for managers

Survey weights are numbers assigned to each respondent that allow us to produce estimates representative of the entire Detroit adult population. Without proper weighting, groups that were oversampled or that had higher response rates would be overrepresented in the estimates.

7.2 Base weights (VDK Ch. 13, Sec. 3)

- The base weight (also called design weight) for a respondent is the inverse of the overall selection probability. At the BG level, the two-stage selection gives a joint probability of $\pi_{ij} = \pi_i \cdot \pi_{j|i}$, so the BG-level base weight is:

$$d_{0,ij} = \frac{1}{\pi_i \cdot \pi_{j|i}}$$

- Under the full three-stage self-weighting design, the person-level probability in domain d would be $\pi_k(d) = f_d$ (VDK p. 283), giving a base weight of $d_0(d) = 1/f_d = N_d/n_d^{\text{select}}$ that is constant within each domain (Lecture Ch. 13, Example 13.3).
- **QC check** (Lecture Slide 20): the sum of base weights across the expected sample in each domain should approximate the domain population total N_d.

7.3 Unknown eligibility (VDK Ch. 13, Sec. 4)

- During field listing, interviewers may encounter housing units whose eligibility cannot be determined (e.g., units that appear occupied but where contact is never made, or addresses that turn out to be vacant or demolished). These cases are classified as unknown eligibility (s_{UNK}).
- The standard approach is to redistribute the weight of unknowns to cases with known eligibility status within adjustment classes formed from frame variables available for all cases (Lecture Slide 30):

$$a_{1b} = \frac{\sum_{i \in s_b} d_{0i}}{\sum_{i \in s_{b,KN}} d_{0i}}$$

  where s_b is all sample cases in class b and $s_{b,KN}$ is the subset whose eligibility is known. The adjusted weight becomes $d_{1i} = a_{1b} \cdot d_{0i}$ for cases in $s_{b,KN}$ and zero for unknowns. Adjustment classes could be formed from tract-level Census demographics (e.g., vacancy rates, housing type).

7.4 Non-response adjustment (VDK Ch. 13, Sec. 5.2; Lecture Slides 45–49)

- Among cases known to be eligible, some will decline to participate. The weighting class NR adjustment within class c is (Lecture Slide 48):

$$a_{2c} = \frac{\sum_{i \in s_{c,E}} d_{1i}}{\sum_{i \in s_{c,ER}} d_{1i}}$$

where $s_{c,E}$ is the set of all eligible cases and $s_{c,ER}$ is the subset of eligible respondents in class c.

- A natural choice for adjustment classes is the four age domains, since response rates are expected to vary substantially (0.35 for ages 18–34 vs. 0.65 for 65+). If the response probability is approximately constant within each domain, this simplifies to $a_{2c} \approx 1/r_d$ (Lecture Slide 50).

8 Variance Estimation (Assignment 5)

8.1 Overview for managers

Because the sample is selected in stages (tracts, then block groups), standard formulas that assume simple random sampling would underestimate the uncertainty of survey estimates. Special variance estimation methods are needed that account for the clustering of persons within tracts. I describe two approaches below: Taylor series linearization (the primary recommendation) and jackknife repeated replication (an alternative).

8.2 Taylor series linearization (primary method)

- For systematic PPS selection, exact joint selection probabilities are not available. The standard practice (VDK Sec. 3.3) is to form **paired analytic strata** by grouping adjacent sample PSUs in the FIPS-sorted selection order. With $m = 40$ sample PSUs, this creates 20 strata, each containing 2 PSUs.
- Within this framework, `survey::svydesign()` uses the ultimate cluster variance estimator (Lecture Ch. 9 Slide 65) to estimate standard errors via Taylor series linearization. This treats each PSU as a "cluster" and captures all within-PSU variability (from both the SSU and person stages) through the between-PSU variation.

```r
# pair adjacent sample PSUs into analytic strata
psu_sample_ve <- psu_sample |>
  arrange(Tract) |>
  mutate(psu_order = row_number(),
         analytic_stratum = ceiling(psu_order / 2))

cat("analytic strata:", max(psu_sample_ve$analytic_stratum), "\n")
```

```
analytic strata: 20
```

```r
psu_sample_ve |> count(analytic_stratum) |> kable()
```

analytic_stratum	n
1	2
2	2
3	2
4	2
5	2
6	2
7	2
8	2
9	2
10	2
11	2
12	2
13	2
14	2
15	2

analytic_stratum	n
16	2
17	2
18	2
19	2
20	2

- The analyst would set up the survey design object as follows after data collection. The `id` identifies PSUs (tracts), `strata` identifies the paired analytic strata, and `weights` are the final NR-adjusted weights. The `nest = TRUE` argument tells `svydesign` that PSU IDs restart within each stratum.

```
# after data collection, set up the tsl design:
detroit_design <- svydesign(
  id = ~ psu_id,
  strata = ~ analytic_stratum,
  weights = ~ final_weight,
  data = survey_data,
  nest = TRUE)

# estimate proportion using public transit as main transport
svymean(~ I(transit == "public"), design = detroit_design)

# domain estimates by age group
svyby(~ I(transit == "public"), ~ age_domain,
      design = detroit_design, svymean)
```

8.3 Jackknife repeated replication (alternative method)

- As an alternative to linearization, delete-one-PSU jackknife (JKn) can be used (VDK Ch. 15). This method systematically drops one PSU at a time, re-estimates the statistic from each reduced sample, and computes the variance from the spread of these $m = 40$ replicate estimates. JKn is especially useful for nonlinear estimators (e.g., ratios, quantiles) where the Taylor approximation may be less accurate.

```
# convert the tsl design to jackknife replicate weights
detroit_jk <- as.svrepdesign(
  design = detroit_design,
  type = "JKn")

# same estimators, variance computed via 40 replicates
svymean(~ I(transit == "public"), design = detroit_jk)
```

9 Maps (Assignment 7)

- Produce maps of the Detroit sampling frame and the selected sample. The `ggmap` versions use Stadia Maps basemap tiles (matching Assignment 7 style) to provide geographic context: roads, water features, and neighborhood boundaries are visible beneath the choropleth layers. The `geom_sf` fallback versions render without an API key.

```
# tigris kables download progress that ignores quarto's
# message/warning suppression, so I capture it manually
invisible(capture.output(suppressMessages({
  detroit_tracts_sf <- tracts(state = "MI", county = "Wayne", year = 2020) |>
    right_join(tract_fips, by = "GEOID") |> st_transform(crs = 4326)

  detroit_bgs_sf <- block_groups(state = "MI", county = "Wayne", year = 2020) |>
    mutate(GEOIDt = str_sub(GEOID, 1, 11)) |>
    right_join(tract_fips, by = c("GEOIDt" = "GEOID")) |>
    select(-GEOIDt) |> st_transform(crs = 4326)
})))
```

```
# detroit bounding box for stadia tiles
detroit_bbox <- c(left = -83.29, bottom = 42.25, right = -82.91, top = 42.45)
detroit_basemap <- get_stadiamap(bbox = detroit_bbox, zoom = 12,
                                 maptype = "stamen_toner_lite")

# shared theme matching assignment 7
map_theme <- theme_void() +
  theme(plot.title = element_text(face = "bold", size = 13, hjust = .5),
        plot.subtitle = element_text(size = 9, hjust = .5, color = "gray40"),
        legend.title = element_text(face = "bold", size = 9),
        legend.text = element_text(size = 8),
        legend.key.height = unit(.8, "cm"),
        legend.key.width = unit(.4, "cm"),
        plot.margin = margin(-5, 4, -5, 4))
```

```
# ggmap: all tracts and block groups on basemap
ggmap(detroit_basemap) +
  geom_sf(data = detroit_bgs_sf, fill = NA,
          linetype = "dotted", linewidth = 0.2, color = "grey50",
          inherit.aes = FALSE) +
  geom_sf(data = detroit_tracts_sf, fill = NA,
          color = "black", linewidth = 0.3, inherit.aes = FALSE) +
  labs(title = "City of Detroit",
       subtitle = "Census Tracts (solid) and Block Groups (dotted)") +
  map_theme
```

Figure 1

```
# ggmap: selected tracts (blue fill) and selected BGs (red outline)
ggmap(detroit_basemap) +
  geom_sf(data = detroit_tracts_sf, fill = NA,
          color = "grey70", linewidth = 0.2, inherit.aes = FALSE) +
  geom_sf(data = detroit_tracts_sf |> filter(GEOID %in% psu_sample$Tract),
          fill = viridis(1, begin = 0.4, alpha = 0.3),
          color = viridis(1, begin = 0.4), linewidth = 0.5,
```

```
        inherit.aes = FALSE) +
geom_sf(data = detroit_bgs_sf |> filter(GEOID %in% sample_bgs$BlockGroup),
        fill = NA, color = "red", linewidth = 0.7, inherit.aes = FALSE) +
labs(title = "Selected Sample: 40 Tracts and Block Groups",
     subtitle = "Blue fill = sample tracts; Red outline = selected BGs") +
map_theme
```

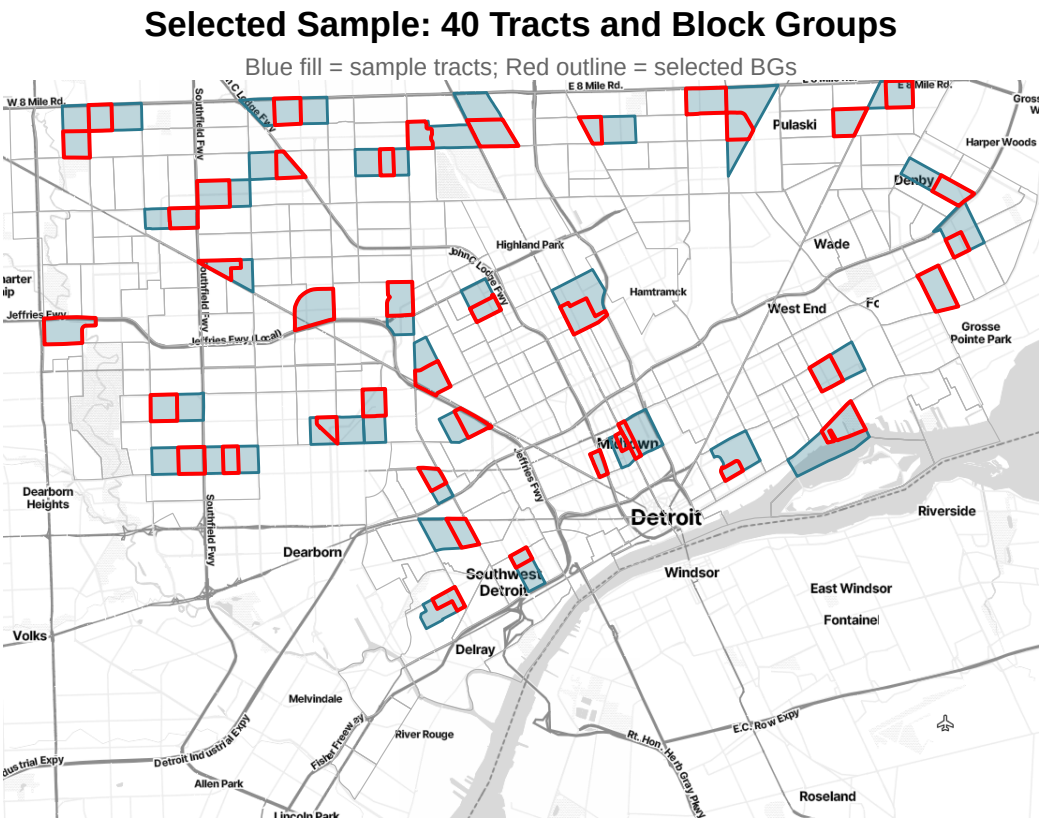

Figure 2

9.0.1 Fallback maps (no API key required)

```
ggplot() +
  geom_sf(data = detroit_bgs_sf, fill = NA,
          linetype = "dotted", linewidth = 0.2, color = "grey50") +
  geom_sf(data = detroit_tracts_sf, fill = NA,
          color = "black", linewidth = 0.4) +
  labs(title = "City of Detroit",
       subtitle = "Tracts (solid) and Block Groups (dotted)") +
  theme_void(base_size = 10)
```

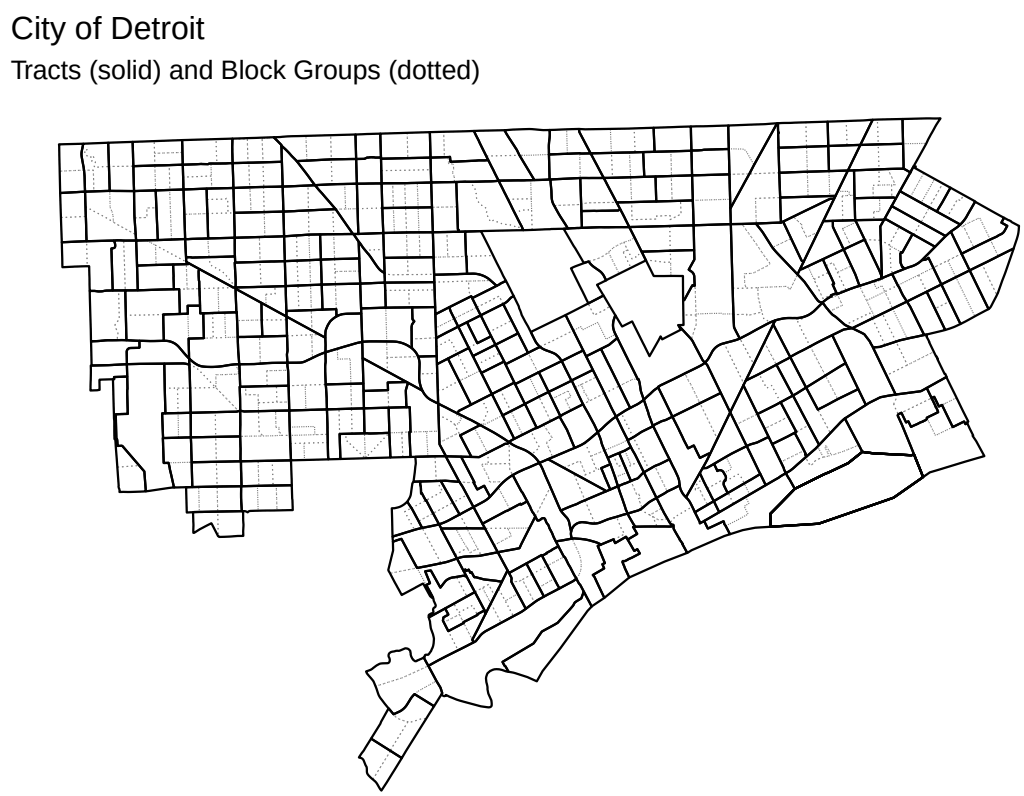

Figure 3

```r
ggplot() +
  geom_sf(data = detroit_tracts_sf, fill = NA,
          color = "grey70", linewidth = 0.3) +
  geom_sf(data = detroit_tracts_sf |> filter(GEOID %in% psu_sample$Tract),
          fill = viridis(1, begin = 0.4, alpha = 0.3),
          color = viridis(1, begin = 0.4), linewidth = 0.6) +
  geom_sf(data = detroit_bgs_sf |> filter(GEOID %in% sample_bgs$BlockGroup),
          fill = NA, color = "red", linewidth = 0.8) +
  labs(title = "Selected Sample: 40 Tracts and Block Groups",
       subtitle = "Blue = sample tracts; Red = selected BGs") +
  theme_void(base_size = 10)
```

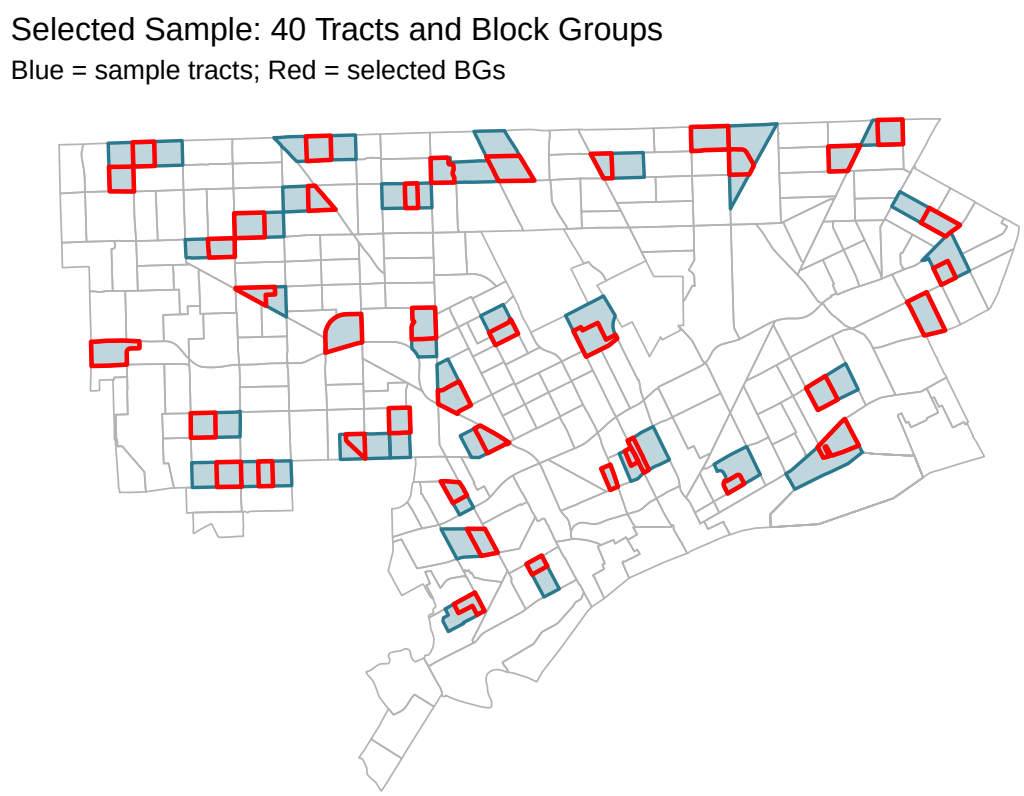

Figure 4

10 Appendix

10.1 A. Sample PSU and SSU listing

- The table below lists all 40 selected BGs with their Census population data, composite MOS, selection probabilities at each stage (π_i and $\pi_{j|i}$), the joint BG probability (π_{ij}), and the domain-specific person sampling rates that field teams would implement. The rates are listed in the `rate_` columns.

```
sample_listing <- sample_bgs |>
  select(Tract, BlockGroup, TotAdult, starts_with("Age"),
         MOS_bg, pi_i, pij_given_i, pi_ij,
         starts_with("rate_"), exp_workload) |>
  arrange(Tract)

# table 1: census data and selection probabilities
sample_listing |>
  select(Tract, BlockGroup, TotAdult, MOS_bg, pi_i, pij_given_i, pi_ij) |>
  mutate(across(where(is.numeric), ~ round(.x, 4))) |>
  kable(booktabs = TRUE, format = "latex")
```

Tract	BlockGroup	TotAdult	MOS_bg	pi_i	pij_given_i	pi_ij
26163500200	261635002002	1480	6.6684	0.1746	0.7090	0.1238
26163500900	261635009002	1419	6.3981	0.2151	0.5523	0.1188
26163501400	261635014002	709	3.1886	0.2241	0.2642	0.0592
26163502000	261635020001	1289	5.8076	0.1078	1.0000	0.1078
26163503400	261635034001	673	3.0255	0.0562	1.0000	0.0562
26163505100	261635051003	1018	4.5506	0.1871	0.4516	0.0845
26163506100	261635061001	1379	6.1868	0.1149	1.0000	0.1149
26163506900	261635069003	674	3.0360	0.2129	0.2647	0.0564
26163508000	261635080001	835	3.7442	0.0695	1.0000	0.0695
26163511400	261635114002	843	3.7546	0.1388	0.5022	0.0697
26163513900	261635139001	612	2.7254	0.0966	0.5239	0.0506
26163515700	261635157001	1663	7.4092	0.3629	0.3790	0.1376
26163516700	261635167001	820	3.6296	0.2155	0.3127	0.0674
26163517500	261635175002	1282	5.6876	0.2080	0.5076	0.1056
26163520300	261635203003	1395	6.2118	0.2077	0.5551	0.1153
26163521800	261635218001	1179	5.2540	0.0975	1.0000	0.0975
26163523300	261635233001	639	2.9078	0.1699	0.3177	0.0540
26163524200	261635242003	1144	5.1839	0.2429	0.3963	0.0962
26163525700	261635257001	946	4.2931	0.2451	0.3253	0.0797
26163526400	261635264002	458	2.0628	0.0808	0.4741	0.0383
26163530800	261635308001	648	2.8942	0.0906	0.5932	0.0537
26163531800	261635318002	551	2.4593	0.0936	0.4876	0.0457
26163533600	261635336001	438	1.9639	0.0864	0.4219	0.0365
26163534700	261635347003	887	3.9723	0.1842	0.4003	0.0737
26163535600	261635356001	531	2.3719	0.2404	0.1832	0.0440
26163536500	261635365001	691	3.1019	0.1332	0.4322	0.0576
26163537200	261635372001	450	2.0288	0.0377	1.0000	0.0377
26163538200	261635382001	905	4.0606	0.1349	0.5590	0.0754
26163538600	261635386001	963	4.3238	0.3010	0.2667	0.0803
26163539200	261635392003	1517	6.7827	0.3245	0.3880	0.1259
26163539600	261635396001	1050	4.6936	0.2030	0.4293	0.0871
26163540400	261635404002	962	4.3008	0.1684	0.4742	0.0798
26163540900	261635409002	1158	5.1879	0.2197	0.4384	0.0963
26163541500	261635415001	1352	6.0412	0.2405	0.4665	0.1122
26163542300	261635423001	667	2.9966	0.1452	0.3832	0.0556
26163543100	261635431001	1037	4.6503	0.1591	0.5426	0.0863
26163544000	261635440001	1985	8.8996	0.1652	1.0000	0.1652
26163545500	261635455001	835	3.7669	0.2170	0.3222	0.0699
26163545900	261635459002	1248	5.6278	0.2118	0.4933	0.1045
26163546700	261635467001	1268	5.6785	0.1741	0.6057	0.1054

```r
# table 2: person-level domain rates and workload
sample_listing |>
  select(Tract, BlockGroup, starts_with("rate_"), exp_workload) |>
```

```r
mutate(
  # shorten fips to last 6 digits for display
  Tract = str_sub(Tract, -6),
  BlockGroup = str_sub(BlockGroup, -7),
  across(where(is.numeric), ~ round(.x, 4))
) |>
kable(booktabs = TRUE, format = "latex") |>
kable_styling(latex_options = c("scale_down", "hold_position"))
```

Tract	BlockGroup	rate_Age18to34	rate_Age35to49	rate_Age50to64	rate_Age65plus	exp_workload
500200	5002002	0.0358	0.0396	0.0343	0.0354	53.8614
500900	5009002	0.0373	0.0413	0.0357	0.0369	53.8614
501400	5014002	0.0748	0.0828	0.0717	0.0740	53.8614
502000	5020001	0.0411	0.0455	0.0394	0.0407	53.8614
503400	5034001	0.0788	0.0873	0.0756	0.0780	53.8614
505100	5051003	0.0524	0.0581	0.0503	0.0519	53.8614
506100	5061001	0.0385	0.0427	0.0370	0.0382	53.8614
506900	5069003	0.0785	0.0870	0.0753	0.0778	53.8614
508000	5080001	0.0637	0.0706	0.0611	0.0631	53.8614
511400	5114002	0.0635	0.0704	0.0609	0.0629	53.8614
513900	5139001	0.0875	0.0969	0.0839	0.0866	53.8614
515700	5157001	0.0322	0.0357	0.0309	0.0319	53.8614
516700	5167001	0.0657	0.0728	0.0630	0.0651	53.8614
517500	5175002	0.0419	0.0464	0.0402	0.0415	53.8614
520300	5203003	0.0384	0.0425	0.0368	0.0380	53.8614
521800	5218001	0.0454	0.0503	0.0435	0.0449	53.8614
523300	5233001	0.0820	0.0908	0.0786	0.0812	53.8614
524200	5242003	0.0460	0.0510	0.0441	0.0455	53.8614
525700	5257001	0.0555	0.0615	0.0533	0.0550	53.8614
526400	5264002	0.1156	0.1281	0.1109	0.1145	53.8614
530800	5308001	0.0824	0.0913	0.0790	0.0816	53.8614
531800	5318002	0.0969	0.1074	0.0930	0.0960	53.8614
533600	5336001	0.1214	0.1345	0.1164	0.1202	53.8614
534700	5347003	0.0600	0.0665	0.0576	0.0594	53.8614
535600	5356001	0.1005	0.1114	0.0964	0.0995	53.8614
536500	5365001	0.0769	0.0852	0.0737	0.0761	53.8614
537200	5372001	0.1175	0.1302	0.1127	0.1164	53.8614
538200	5382001	0.0587	0.0651	0.0563	0.0581	53.8614
538600	5386001	0.0551	0.0611	0.0529	0.0546	53.8614
539200	5392003	0.0352	0.0389	0.0337	0.0348	53.8614
539600	5396001	0.0508	0.0563	0.0487	0.0503	53.8614
540400	5404002	0.0554	0.0614	0.0532	0.0549	53.8614
540900	5409002	0.0460	0.0509	0.0441	0.0455	53.8614
541500	5415001	0.0395	0.0437	0.0379	0.0391	53.8614
542300	5423001	0.0796	0.0882	0.0763	0.0788	53.8614
543100	5431001	0.0513	0.0568	0.0492	0.0508	53.8614
544000	5440001	0.0268	0.0297	0.0257	0.0265	53.8614
545500	5455001	0.0633	0.0701	0.0607	0.0627	53.8614
545900	5459002	0.0424	0.0469	0.0406	0.0420	53.8614
546700	5467001	0.0420	0.0465	0.0403	0.0416	53.8614

10.2 B. Export files

```
write_csv(psu_frame, here("outputs", "FRAME_PSU_EffectSize.csv"))
write_csv(ssu_frame, here("outputs", "FRAME_BG_EffectSize.csv"))
write_csv(sample_listing, here("outputs", "SAMPLE_EffectSize.csv"))
```

10.3

10.4 C. AI Disclosure

This project utilized Claude (Anthropic, Opus) as an AI coding assistant. The tool was used to debug R code throughout development, including diagnosing vector recycling issues with `across()` when multiplying domain columns by the `f_d` vector, resolving `rename()` conflicts caused by duplicate column names after the PSU linking step, and troubleshooting `kable()` and `kableExtra` table formatting for PDF output such as fitting wide tables using `scale_down` and splitting listings into separate display tables. Claude also assisted with map rendering, suppressing verbose `tigris` download output during Quarto rendering and configuring `ggmap` basemap layers with `inherit.aes = FALSE` for proper `geom_sf` overlay alignment. All AI outputs were reviewed, tested, and modified by team members before inclusion.